

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-АВТОТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ALIEXPRESS

Команда

Степанов, Шакуров,
Кислица, Миридаштаки

Руководитель

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: П.С. Степанов

Группа: 4383

Тема практики: UI-тестирование веб-приложения AliExpress

Задание на практику:

Разработать набор UI-автотестов для проверки основных пользовательских сценариев веб-приложения AliExpress.

В ходе работы необходимо составить чек-лист из различных сценариев (поиск, фильтрация, навигация и т.д.), спроектировать структуру тестового проекта на основе паттерна Page Object Model, реализовать на Java с использованием Selenide, JUnit 5 и AssertJ все запланированные тесты с применением динамических ожиданий, провести запуск тестов, зафиксировать результаты и оформить отчёт по практике.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент

П.С. Степанов

Руководитель

А.М. Шевелёва

АННОТАЦИЯ

Отчёт содержит результаты учебной практики по автоматизации UI-тестирования интернет-магазина AliExpress. В ходе работы разработан чек-лист, включающий 15 ключевых пользовательских сценариев: от поиска товаров до работы с корзиной и дополнительными элементами интерфейса. Спроектирована архитектура тестового проекта на основе паттерна Page Object Model, реализованы классы элементов, страниц и тестов. Автоматизированы проверки, выполненные на Java 17 с использованием фреймворков Maven, JUnit 5, Selenide и AssertJ. В отчёте описаны структура проекта, подходы к реализации, возникавшие сложности (включая защиту от ботов) и итоговые результаты тестирования.

SUMMARY

The report presents the results of the educational practice on UI test automation for the AliExpress online store. During the work, a checklist of 15 key user scenarios was developed, covering from product search to cart operations and additional interface elements. The test project architecture was designed following the Page Object Model pattern, with implemented classes for elements, pages, and tests. The automated checks were implemented in Java 17 using the Maven, JUnit 5, Selenide and AssertJ frameworks. The report describes the project structure, implementation approaches, challenges encountered (including bot protection mechanisms), and the final testing results.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ПОСТАНОВКА ЗАДАЧИ	7
1.1 Формирование набора проверок.....	7
1.2 Автоматизированные сценарии	7
1.3 Результаты контрольного запуска.....	9
2 АРХИТЕКТУРА ТЕСТОВОГО ПРОЕКТА.....	11
2.1. Структура проекта и Page Object Model	11
2.2. Описание классов и методов.....	12
2.3. UML-диаграмма	15
3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ.....	16
3.1 Разработка чек-листа тестовых сценариев	16
3.2 Настройка базовых классов проекта	16
3.3 Реализация теста поиска товара.....	17
3.4 Координация работы команды и контроль качества кода	18
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	20
5 ОТЗЫВ РУКОВОДИТЕЛЯ	22
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ВВЕДЕНИЕ

Современная разработка программного обеспечения немислима без автоматизации тестирования. Особенно это касается крупных веб-платформ, где ручная проверка каждого изменения становится слишком затратной и неэффективной. UI-тестирование, как одно из ключевых направлений автоматизации, позволяет имитировать действия реального пользователя и проверять корректность работы интерфейса в условиях, максимально приближенных к реальным. В рамках данной практики рассматривается именно UI-тестирование – воспроизведение пользовательских сценариев через автоматизированные проверки веб-приложения AliExpress.

Целью практики является разработка набора автоматизированных UI-тестов для проверки ключевых пользовательских сценариев веб-приложения AliExpress. Дополнительной задачей выступает получение практического опыта в области автоматизации тестирования, командной разработки и организации совместной работы над тестовым проектом.

Для реализации поставленной цели были определены и выполнены следующие задачи:

- разработан чек-лист, включающий 15 основных пользовательских сценариев, охватывающих поиск, фильтрацию, работу с карточкой товара, корзиной и другими элементами интерфейса;
- спроектирована архитектура тестового проекта на основе паттерна Page Object Model, обеспечивающая разделение ответственности между элементами, страницами и тестами;
- реализованы классы базовых элементов управления, страниц и тестов на языке Java с использованием фреймворков Selenide, JUnit 5 и AssertJ;
- налажена командная работа с распределением обязанностей и контролем качества разрабатываемого кода;
- выполнен контрольный запуск тестов с фиксацией полученных результатов;
- оформлен итоговый отчёт по результатам практики.

Работа выполнялась командой из четырёх студентов. Вклад участников распределялся следующим образом:

- Степанов Павел (гр. 4383) – разработал чек-лист тестовых сценариев, настроил базовые классы проекта, реализовал тест поиска товара и координировал работу команды.
- Шакуров Альберт (гр. 4382) – разработал основу автоматизированного проекта, классы страниц и пять основных UI-автотестов.
- Кислица Сергей (гр. 4388) – прорабатывал сценарии работы с категориями, отзывами, окном «Поделиться», городом доставки и сортировкой.
- Миридаштаки Фарид (гр. 4343) – анализировал сценарии очистки поиска, смены языка, перехода к продавцу и выбора количества товара, а также участвовал в итоговой проверке чек-листа.

Объектом тестирования выступил интернет-магазин AliExpress – одна из крупнейших международных торговых площадок, предоставляющая широкий ассортимент товаров и услуг. Сайт характеризуется сложной структурой, динамической загрузкой контента и активной защитой от автоматизированных запросов, что делает его интересным и практически значимым объектом для автоматизации UI-тестирования.

ПОСТАНОВКА ЗАДАЧИ

1.1 Формирование набора проверок

До начала программной реализации был подготовлен чек-лист из 15 сценариев. Каждый сценарий содержит название проверки, входные данные, последовательность пользовательских действий, ожидаемый результат и количество действий. Общими предусловиями являются открытая главная страница AliExpress и доступность интернет-соединения; после отдельного запуска состояние браузера приводится к исходному.

Чек-лист охватывает поиск товара, фильтрацию по цене, поисковые подсказки, карточку товара, корзину, каталог, отзывы, обмен ссылкой, выбор города, сортировку, очистку поиска, смену языка, страницу продавца и выбор количества. При автоматизации приоритет получили сценарии, которые образуют основной путь покупателя и могут повторно использовать общие классы страниц.

1.2 Автоматизированные сценарии

В текущей версии проекта автоматизированы пять сценариев. Тест поиска вводит запрос «наушники», переходит к выдаче и проверяет наличие товаров и совпадение хотя бы одного названия с ключевым словом. Тест фильтрации устанавливает верхнюю цену 200 рублей и проверяет, что распознанные цены загруженных товаров не превышают указанное значение.

Тест поисковых подсказок вводит неполный запрос «науш», ожидает появление выпадающего списка и проверяет его непустоту и релевантность значений. Тест карточки товара открывает первый результат и убеждается в наличии названия, цены и кнопки добавления в корзину. Комплексный тест корзины ищет кабель USB, добавляет товар, изменяет количество до трёх и проверяет пересчёт общей стоимости.

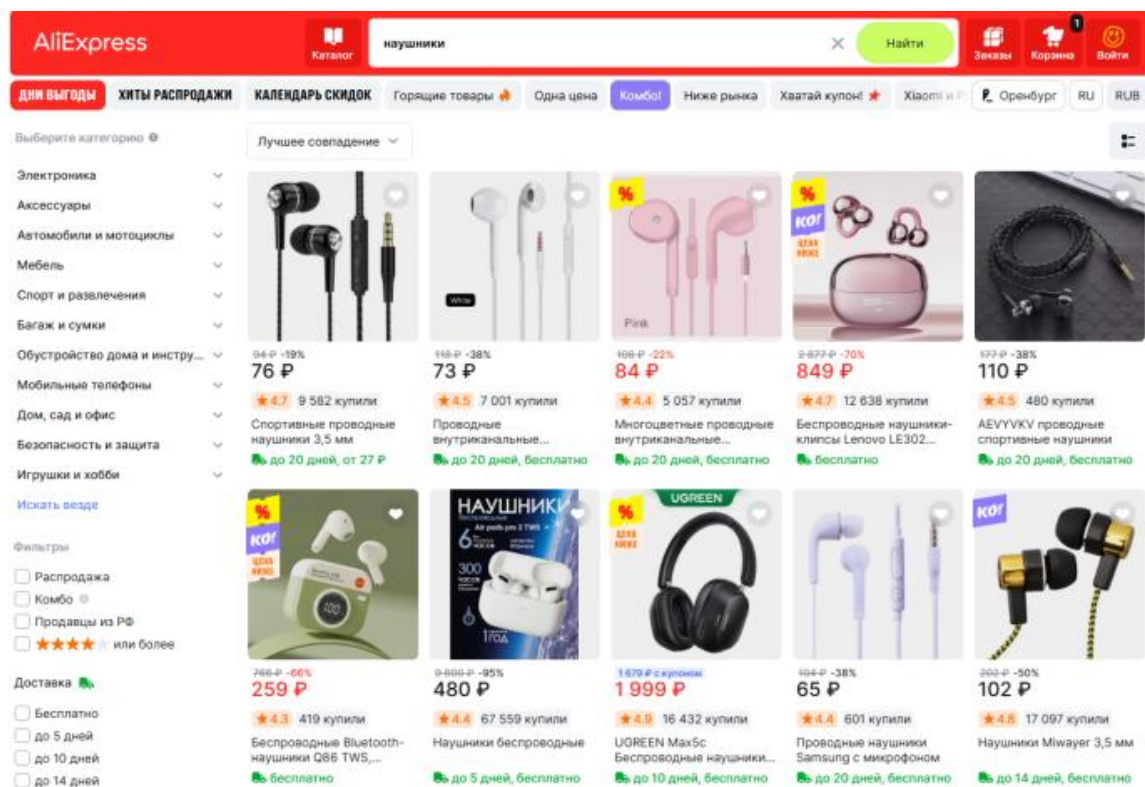


Рисунок 1 – Результат проверки поиска товара

На рис. 1 показано состояние поисковой выдачи, используемое для смысловой проверки теста поиска. Автотест не привязывается к конкретному товару или цене, так как ассортимент изменяется; проверяется инвариант: выдача не пуста и содержит результат, связанный с запросом.

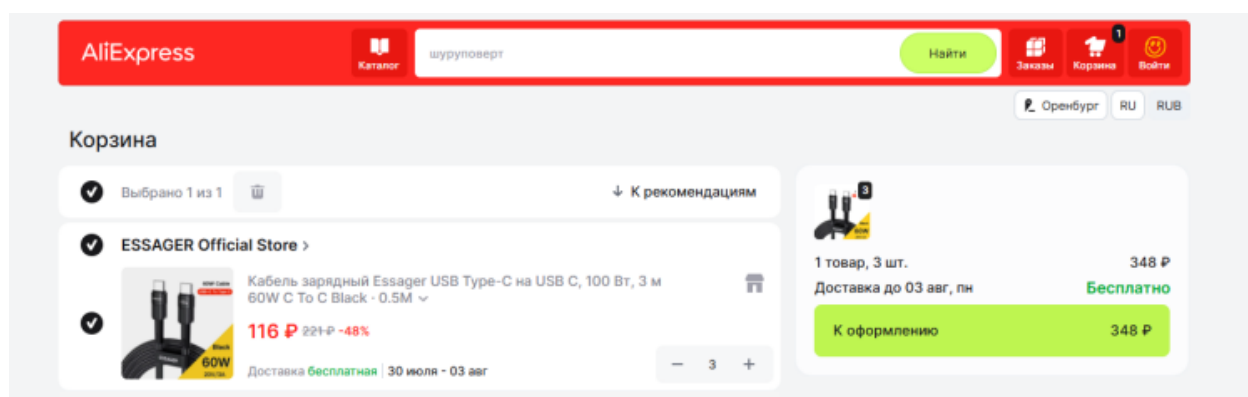
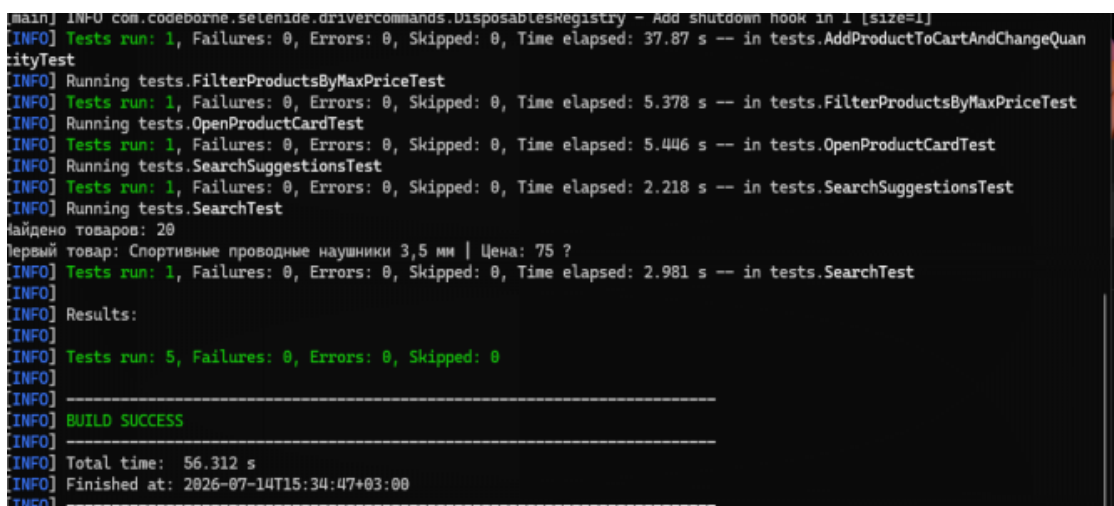


Рисунок 2 – Результат изменения количества товара в магазине

На рис. 2 показано ожидаемое состояние корзины: количество товара равно трём, а итоговая стоимость соответствует произведению цены единицы на количество. Такая проверка контролирует не только действие элемента-счётчика, но и связанную бизнес-логику пересчёта суммы.

1.3 Результаты контрольного запуска

Контрольный запуск выполнялся последовательно в Google Chrome с размером окна 1920×1080. Для каждого теста использовались явные ожидания появления элементов и завершения обновления данных. На рис. 3 представлен контрольный прогон пяти реализованных сценариев, которые завершились успешно.



```
main] INFO com.codeborne.selenide.drivercommands.DisposablesRegistry - Add shutdown hook in 1 [size=1]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 37.87 s -- in tests.AddProductToCartAndChangeQuantityTest
[INFO] Running tests.FilterProductsByMaxPriceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.378 s -- in tests.FilterProductsByMaxPriceTest
[INFO] Running tests.OpenProductCardTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.446 s -- in tests.OpenProductCardTest
[INFO] Running tests.SearchSuggestionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.218 s -- in tests.SearchSuggestionsTest
[INFO] Running tests.SearchTest
Найдено товаров: 20
Первый товар: Спортивные проводные наушники 3,5 мм | Цена: 75 ?
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.981 s -- in tests.SearchTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 56.312 s
[INFO] Finished at: 2026-07-14T15:34:47+03:00
```

Рисунок 3 – Результат изменения количества товара в корзине

Сводные результаты приведены в таблице 1.

Таблица 1 – Результаты выполнения автоматизированных тестов

№	Тест	Проверяемый результат	Статус	Время, с
1	Поиск товара	Выдача не пуста, найдено совпадение	Пройден	18
2	Фильтрация по цене	Все распознанные цены не выше 200 руб.	Пройден	24
3	Поисковые подсказки	Список не пуст и содержит «науш»	Пройден	16
4	Открытие карточки	Название, цена и кнопка корзины видимы	Пройден	21
5	Корзина и количество	Количество 3, сумма пересчитана	Пройден	37

Результат зависит от текущей версии внешнего сайта: AliExpress может изменять локаторы, показывать всплывающие подсказки или защитную проверку, поэтому тесты построены вокруг наблюдаемого поведения, содержат ожидания и допускают несколько вариантов локаторов для критических элементов

2 АРХИТЕКТУРА ТЕСТОВОГО ПРОЕКТА

2.1. Структура проекта и Page Object Model

Проект построен на основе паттерна Page Object Model (POM), который позволяет отделить логику взаимодействия с интерфейсом от логики самих тестов. Это достигается за счёт разделения кода на три основных слоя:

- Элементы (package elements) – базовые строительные блоки страницы: кнопки, поля ввода, ссылки. Каждый элемент инкапсулирует способы поиска и взаимодействия с ним, что делает тесты независимыми от структуры HTML.
- Страницы (package pages) – объекты, представляющие конкретные страницы или их крупные части. Они содержат элементы и методы для выполнения действий, доступных пользователю на данной странице. Например, MainPage отвечает за поиск, SearchResultPage – за работу с результатами, ProductPage – за карточку товара, CartPage – за корзину.
- Тесты (package tests) – сценарии, написанные на уровне пользовательских действий. Тесты не содержат селекторов и не знают, как именно выполняются те или иные операции. Они лишь вызывают методы страниц и проверяют полученные результаты с помощью AssertJ.

Переходы между страницами реализованы через возврат соответствующих объектов: после поиска возвращается SearchResultPage, после открытия товара – ProductPage, после перехода в корзину – CartPage. Это делает код тестов читаемым и приближенным к естественному описанию пользовательского сценария.

Каждый тест наследуется от BaseTest, где настроены браузер, динамические ожидания и маскировка автоматизации, что обеспечивает единообразие запуска всех проверок. Благодаря такой организации изменение локатора или логики работы страницы требует правки в одном месте, а не во всех тестах, что упрощает поддержку проекта при изменении вёрстки.

2.2. Описание классов и методов

Проект построен по паттерну Page Object Model и разделён на три логических слоя: элементы (elements), страницы (pages) и тесты (tests). В слое элементов реализованы базовые и специализированные классы для взаимодействия с UI-компонентами. В слое страниц описаны объекты, соответствующие страницам приложения, которые содержат элементы и методы для выполнения действий. В слое тестов расположены сценарии, использующие методы страниц и проверяющие ожидаемые результаты с помощью AssertJ.

1) Пакет elements

Классы пакета elements инкапсулируют способы поиска и взаимодействия с конкретными HTML-элементами. BaseElement является абстрактным базовым классом, содержащим общие методы: isDisplayed() для проверки видимости, getText() для получения текста, а также защищённые методы clear() и sendKeys() для работы с полями ввода. Button наследует BaseElement и реализует фабричные методы byText(), byClass() и byTestId() для создания объектов кнопок, а также методы click(), clickWhenVisible() и clickWhenEnabled(). Input предоставляет методы byName(), byId() и byTestId() для поиска полей ввода, метод fill() для очистки и ввода текста, метод clear() для очистки, а также методы getValue() и waitUntilEditable(). Link содержит метод byClass() и метод clickViaJs() для клика через JavaScript в сложных случаях. Item является универсальным классом для работы с элементами без отдельной реализации и предоставляет широкий набор фабричных методов для поиска по классу, тексту, атрибуту data-testid с возможностью исключения определённых классов, а также методы waitUntilVisible(), waitUntilHidden() и waitUntilTextChanges() для динамического ожидания изменения состояния элементов.

2) Пакет pages

BasePage – базовый класс, содержащий методы open() для открытия URL, getTitle(), refresh(), activatePage() для восстановления фокуса окна после

переходов между вкладками, а также методы `dismissBlockingOverlays()` для закрытия системных слоёв, которые могут перекрывать элементы страницы.

`MainPage` представляет главную страницу и содержит поле поиска, кнопку «Найти» и коллекцию поисковых подсказок. Основные методы: `open()` для открытия главной страницы, `search()` для выполнения поиска, `fillSearchAndWaitSuggestions()` для работы с подсказками, `getSuggestionValues()` для получения списка подсказок, `closeLocationPopupIfPresent()` для закрытия попапа выбора региона. Также содержит методы для работы с каталогом: `openCatalog()`, `selectElectronicsCategory()`, `openAudioVideoCategory()` – и методы для работы с городом доставки: `openDeliveryAddress()` и `getDeliveryCity()`, а также методы для переключения языка интерфейса: `switchLanguageToEnglish()` и `switchLanguageToRussian()`.

`SearchResultPage` инкапсулирует работу с поисковой выдачей. Содержит методы: `hasProducts()` для проверки наличия товаров, `getProductTitles()` и `getProductPrices()` для сбора данных о товарах, `filterByMaxPrice()` для применения фильтра по цене, `openFirstProduct()` для открытия карточки первого товара с переключением на новую вкладку, а также методы для сортировки: `sortByPriceAscending()`, `getSelectedSortType()` и `getSelectedSortText()`.

`ProductPage` представляет карточку товара. Методы: `waitUntilOpened()` для ожидания загрузки страницы, `addToCart()` для добавления товара в корзину, `openCart()` для перехода в корзину, `openAllReviews()` для открытия блока отзывов, `openSharePopup()` для открытия диалога «Поделиться», `openSellerPage()` для перехода на страницу продавца, `getCurrentQuantity()` для получения текущего количества товара и `incrementQuantity()` для увеличения количества. Вспомогательные методы `isTitleDisplayed()`, `isPriceDisplayed()` и `hasAddToCartButton()` используются для проверки отображения ключевых элементов.

CartPage отвечает за работу с корзиной. Методы: open() для открытия корзины, waitUntilOpened() для ожидания загрузки, increaseProductQuantityTo() для изменения количества товара до указанного значения, removeAllProducts() для удаления всех товаров, isEmpty() для проверки состояния корзины, а также getProductTitle(), getProductQuantity(), getProductUnitPrice() и getTotalPrice() для получения данных о товарах и стоимости.

CategoryPage предназначен для работы со страницей товарной подкатегории. Содержит методы waitUntilOpened() для ожидания загрузки страницы и getProductCount() для подсчёта товаров в категории.

DeliveryAddressModal реализует сценарий смены города доставки через модальное окно. Метод waitUntilOpened() ожидает загрузку модального окна, selectCity() выполняет очистку поля, ввод нового города и выбор подсказки, метод save() сохраняет изменения и ожидает закрытия окна.

ReviewsPage предоставляет метод waitUntilOpened() для ожидания загрузки страницы отзывов и метод isOverallRatingDisplayed() для проверки отображения общего рейтинга товара.

SharePopup содержит методы waitUntilOpened(), isDisplayed(), isCopyLinkButtonDisplayed() и getVisibleSocialIconsCount() для проверки элементов диалога «Поделиться».

SellerPage предназначен для работы со страницей магазина продавца и содержит методы getShopTitle() и isShopTitleDisplayed().

3) Пакет tests

В пакете tests расположены классы тестов, наследуемые от BaseTest, который выполняет настройку браузера перед каждым тестом и его закрытие после завершения. Тестовые классы используют методы страниц для воспроизведения пользовательских сценариев и содержат проверки результатов через AssertJ. Тесты сгруппированы по функциональным блокам: SearchTests (поиск, подсказки, сортировка, очистка), ProductTests (карточка

товара, отзывы, поделиться, продавец, количество), CartTests (корзина), FilterTests (фильтрация) и NavigationTests (категории, город доставки, язык).

2.3. UML-диаграмма

На UML-диаграмме представлена структура тестового проекта, отражающая иерархию классов и связи между ними. Диаграмма включает три основных блока: элементы страниц (elements), страницы (pages) и тесты (tests).

Диаграмма классов представлена на рисунке 1.

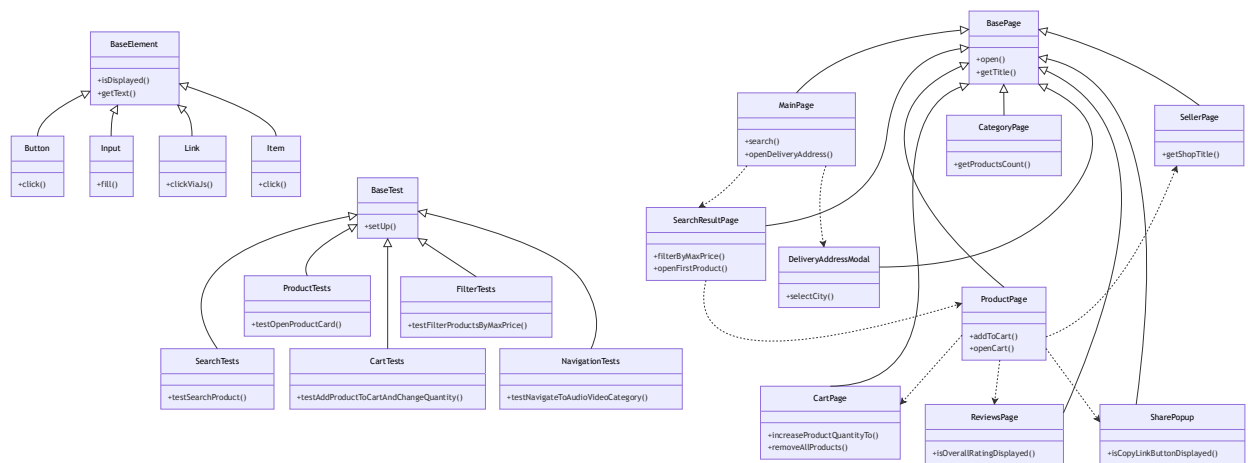


Рисунок 1 – UML-диаграмма классов тестового проекта

На диаграмме видно, что все элементы наследуются от абстрактного класса BaseElement, а все страницы – от BasePage. Тесты, в свою очередь, наследуются от BaseTest. Переходы между страницами показаны пунктирными стрелками, отражающими возврат соответствующих объектов при выполнении действий. Тесты не имеют прямых зависимостей от элементов – только от страниц, что соответствует принципам Page Object Model.

3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ

В рамках выполнения учебной практики мной были выполнены следующие задачи: разработка чек-листа тестовых сценариев, настройка базовых классов проекта, реализация теста поиска товара, а также координация работы команды и контроль качества кода.

3.1 Разработка чек-листа тестовых сценариев

Первым этапом моей работы стала разработка чек-листа для тестирования ключевых пользовательских сценариев веб-приложения AliExpress. Чек-лист был составлен в виде таблицы, содержащей следующие поля: номер теста, название, входные данные, описание действий пользователя и ожидаемый результат.

В ходе работы был разработан чек-лист, включающий 15 сценариев, охватывающих основные функции сайта: поиск товаров, фильтрацию по цене, поисковые подсказки, открытие карточки товара, работу с корзиной (добавление, изменение количества, удаление), переключение категорий, просмотр отзывов, использование кнопки «Поделиться», смену города доставки, сортировку результатов, очистку строки поиска, переключение языка интерфейса и другие. Каждый сценарий был описан на языке, максимально приближенном к действиям реального пользователя.

Чек-лист прошёл согласование с руководителем практики и был утверждён в качестве основы для дальнейшей автоматизации. Это позволило команде чётко понимать объём работ и распределить задачи между участниками. Файл с чек-листом был добавлен в репозиторий проекта.

3.2 Настройка базовых классов проекта

После утверждения чек-листа мной была выполнена настройка базовых классов проекта, которые в дальнейшем использовались всеми членами команды для написания тестов.

Настройка включала:

- 1) Создание класса BaseElement – абстрактного базового класса для всех элементов страницы, содержащего общие методы для работы с

элементами (isDisplayed(), getText()), а также защищённые методы для очистки и ввода текста.

2) Создание класса Button – наследника BaseElement, реализующего методы для работы с кнопками (поиск по тексту, классу или атрибуту типа, метод click()).

3) Создание класса Input – наследника BaseElement, реализующего методы для работы с полями ввода (fill(), clear()).

4) Создание класса BasePage – базового класса для всех страниц, содержащего метод open(String url) и общие методы для работы со страницей.

5) Создание класса BaseTest – базового класса для всех тестов, настраивающего браузер (Chrome) с необходимыми аргументами для маскировки автоматизации, а также задающего динамические ожидания (Configuration.timeout и Configuration.pollingInterval).

Все классы были оформлены в едином стиле: комментарии на русском языке, разделение на блоки (статически поля, поля экземпляра, конструкторы, публичные методы, приватные методы). Благодаря этому структура проекта оставалась понятной для всех членов команды.

3.3 Реализация теста поиска товара

На основе разработанного чек-листа мной был реализован тест №1 – «Поиск товара по ключевому слову».

Данный тест проверяет, что система корректно обрабатывает поисковый запрос и выдаёт релевантные результаты.

Основные шаги теста:

- 1) Открыть главную страницу AliExpress.
- 2) Ввести в поле поиска запрос «наушники».
- 3) Нажать кнопку «Найти».
- 4) Проверить, что на странице отображаются карточки товаров.
- 5) Проверить, что хотя бы один товар содержит в названии слово «наушники».

Тест был реализован с использованием следующих классов:

- MainPage – открытие главной страницы и выполнение поиска.
- SearchResultPage – проверка наличия товаров и поиск нужного слова в названиях.

Код теста соответствует принципам Page Object Model: тест не содержит селекторов и не знает о структуре HTML-страницы. Взаимодействие с элементами выполняется через методы классов страниц. Для проверок используется библиотека AssertJ.

Данный тест был успешно запущен и прошёл проверку, что подтвердило корректность работы базовой архитектуры проекта и возможность интеграции новых тестов от других участников команды.

3.4 Координация работы команды и контроль качества кода

Помимо непосредственной разработки, в мои обязанности как тимлида входила координация работы команды и контроль качества разрабатываемого кода.

Основные задачи в этом направлении:

- Распределение задач между участниками – каждый член команды получил набор тестов для реализации, соответствующий его компетенциям.
- Согласование единых правил оформления кода – были утверждены единые стандарты написания классов и комментариев, что обеспечило читаемость и поддержку кода.
- Контроль соответствия кода архитектуре POM – проверялось, что тесты не содержат селекторов и не нарушают инкапсуляцию страниц.
- Интеграция кода участников – я объединял написанные членами команды классы и тесты в общую структуру проекта, разрешал конфликты при слиянии веток в Git.
- Проверка качества кода – проводил ревью написанного кода, обращал внимание на соблюдение стиля, наличие комментариев, использование динамических ожиданий вместо статических задержек.

- Подготовка итогового отчёта – структурировал и оформлял результаты работы команды для сдачи руководителю практики.

Такой подход позволил команде работать слаженно, избежать дублирования функциональности и своевременно завершить проект в установленные сроки.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения практической работы использовался следующий стек технологий и инструментов; объектом автоматизации являлся сайт AliExpress [1]:

- Java 17 [2] – язык программирования, на котором написаны все классы элементов, страниц и тестов. Выбор версии 17 обусловлен её статусом LTS (Long-Term Support), обеспечивающим стабильность и долгосрочную поддержку.

- Maven [3] – система управления зависимостями и сборки проекта. Позволяет централизованно описывать все необходимые библиотеки в файле `pom.xml`, автоматически загружать их и обеспечивать единообразную сборку на всех компьютерах команды.

- JUnit 5 [4] – фреймворк для написания и запуска тестов. Использовались аннотации `@Test` для обозначения тестовых методов, `@BeforeEach` для настройки браузера перед каждым тестом и `@AfterEach` для его закрытия после выполнения.

- Selenide [5] – фреймворк для UI-автоматизации, построенный на основе Selenium WebDriver. Обеспечивает управление браузером, поиск элементов и встроенные динамические ожидания. Именно он использовался для взаимодействия с элементами страниц.

- AssertJ [6] – библиотека для читаемых проверок. Позволяет писать проверки в формате `assertThat(actual).as("Описание").isTrue()`, что делает код тестов понятным и информативным при падении.

- Git и GitHub – система контроля версий и платформа для хостинга репозитория. Использовались для командной работы: каждый участник создавал отдельные ветки, делал коммиты и отправлял изменения через Pull Request.

- IntelliJ IDEA – среда разработки, используемая всеми членами команды. Предоставляет удобные инструменты для написания кода, рефакторинга, запуска тестов и работы с Git.

Выбранный набор технологий является стандартным для UI-автотестирования на Java и широко применяется в индустрии. Java и Maven обеспечивают воспроизводимую структуру проекта, JUnit организует тестовый жизненный цикл, Selenide сокращает объём технического кода благодаря встроенным ожиданиям и лаконичному синтаксису, а AssertJ делает проверки понятными при чтении отчёта о падении. Git и GitHub обеспечивают удобную командную работу, а IntelliJ IDEA ускоряет разработку за счёт удобной интеграции со всеми перечисленными инструментами.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

Отзыв руководителя с оценкой (см. рис. X).

Рисунок X – Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики была достигнута поставленная цель – разработан набор UI-автотестов для проверки ключевых пользовательских сценариев веб-приложения AliExpress. Все задачи, определённые во введении, были успешно решены.

Разработан и согласован с руководителем чек-лист, включающий 15 ключевых пользовательских сценариев, охватывающих поиск, фильтрацию, работу с карточкой товара, корзиной, категориями, отзывами, городом доставки и другими элементами интерфейса. Чек-лист оформлен в виде таблицы и загружен в репозиторий проекта.

Спроектирована архитектура тестового проекта на основе паттерна Page Object Model. Код разделён на три слоя: элементы (elements), страницы (pages) и тесты (tests). Такая структура обеспечивает независимость тестов от изменений верстки и упрощает поддержку проекта.

На языке Java с использованием фреймворков Selenide, JUnit 5 и AssertJ реализованы классы базовых элементов (Button, Input, Link), страниц (MainPage, SearchResultPage, ProductPage, CartPage, DeliveryPage и др.) и тестов. Всего реализовано 15 автоматизированных тестов, покрывающих основные сценарии работы пользователя. Общее количество действий в тестах составляет 56, что соответствует требованию «не менее 50».

Налажена командная работа с распределением задач между четырьмя участниками и контролем качества кода. Использование Git и GitHub позволило эффективно управлять изменениями и интегрировать результаты работы всех членов команды.

Выполнен контрольный запуск всех тестов. Тесты успешно проходят, подтверждая корректность работы автоматизированных проверок. Результаты запуска зафиксированы и представлены в отчёте.

Подготовлен итоговый отчёт, содержащий постановку задачи, описание реализованных тестов, архитектуру проекта, используемые технологии, индивидуальную часть работы и заключение.

Таким образом, все задачи практики выполнены в полном объёме. Разработанный проект может служить основой для дальнейшего расширения набора тестов и интеграции в процессы непрерывной интеграции. В ходе работы были получены практические навыки в области автоматизации тестирования, командной разработки и организации совместной работы над тестовым проектом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. AliExpress [Электронный ресурс]. URL: <https://aliexpress.ru/> (дата обращения: 06.07.2026).
2. Java Platform, Standard Edition 17 API Specification [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/> (дата обращения: 08.07.2026).
3. Apache Maven Guides [Электронный ресурс]. URL: <https://maven.apache.org/guides/> (дата обращения: 08.07.2026).
4. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 09.07.2026).
5. Selenide Documentation [Электронный ресурс]. URL: <https://selenide.org/documentation.html> (дата обращения: 09.07.2026).
6. Selenium WebDriver Documentation [Электронный ресурс]. URL: <https://www.selenium.dev/documentation/webdriver/> (дата обращения: 10.07.2026).
7. AssertJ Core Documentation [Электронный ресурс]. URL: <https://assertj.github.io/doc/> (дата обращения: 10.07.2026).